PHPUnit 9.6 with Zend Framework 1

Practical offline testing guide for PHP 7.4 and MariaDB

An original training guide with runnable examples and adaptation notes. This is not the official PHPUnit manual. October 2026.

Contents: fundamentals; installation; assertions; fixtures; exceptions; data providers; mocks; ZF1 bootstrap and controller testing; MariaDB integration tests; JWT testing; coverage; troubleshooting; test checklist.

1. What PHPUnit does

PHPUnit is a framework that runs PHP test methods and reports whether their assertions pass. A test is a repeatable check of one expected behavior. Unit tests isolate logic; integration tests use real dependencies such as MariaDB; functional tests exercise a request/controller boundary.

```
class Calculator {
    public function add($a, $b) { return $a + $b; }
}

use PHPUnit\Framework\TestCase;
class CalculatorTest extends TestCase {
    public function testAddition(): void {
        $calc = new Calculator();
        $this->assertSame(5, $calc->add(2, 3));
    }
}
```

assertSame checks both value and type. assertEquals allows some type coercion. Prefer assertSame for precise application contracts.

2. Compatibility and installation

PHPUnit 9.6 supports PHP 7.3 or newer and is appropriate for PHP 7.4. ZF1 is a legacy framework, so confirm the exact framework revision and PHP compatibility in your environment. For offline production systems, run tests in a separate development or CI environment rather than installing test tooling on production.

```
# Development workstation (with Composer available)
composer require --dev phpunit/phpunit:^9.6
php vendor/bin/phpunit --version

# Windows
vendor\bin\phpunit --version

# Run tests
php vendor/bin/phpunit --testsuite Unit
```

If using an offline workstation, resolve Composer dependencies on an internet-connected machine with the same PHP version/extensions, then transfer composer.json, composer.lock, and the complete vendor directory. Do not run tests against production credentials.

3. Suggested project layout

```
project/
  application/
    Bootstrap.php
    configs/application.ini
    modules/api/controllers/AuthController.php
  library/User/
    JwtService.php
  tests/
    bootstrap.php
    Unit/CalculatorTest.php
    Integration/UserRepositoryTest.php
    Functional/AuthControllerTest.php
  vendor/
  phpunit.xml
```

4. Basic PHPUnit configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="tests/bootstrap.php"
         colors="true" failOnWarning="true">
  <testsuites>
    <testsuite name="Unit">
      <directory>tests/Unit</directory>
```

```
    </testsuite>
    <testsuite name="Integration">
      <directory>tests/Integration</directory>
    </testsuite>
    <testsuite name="Functional">
      <directory>tests/Functional</directory>
    </testsuite>
  </testsuites>
</phpunit>
```

Set the environment explicitly in tests/bootstrap.php and load only testing configuration. Be aware that PHPUnit XML options can vary by minor release.

5. Test bootstrap for ZF1

```
<?php
// tests/bootstrap.php
if (!defined('APPLICATION_ENV')) {
    define('APPLICATION_ENV', 'testing');
}
if (!defined('APPLICATION_PATH')) {
    define('APPLICATION_PATH', realpath(__DIR__ . '/../application'));
}
require dirname(__DIR__) . '/vendor/autoload.php';
set_include_path(implode(PATH_SEPARATOR, array(
    dirname(__DIR__) . '/library',
    dirname(__DIR__) . '/library/Zend', // adjust actual ZF1 path
    get_include_path()
)));
require_once 'Zend/Loader/Autoloader.php';
Zend_Loader_Autoloader::getInstance()->registerNamespace('User_');
```

Adjust the Zend framework library path to your actual deployment. If the Zend directory itself contains the Zend/ subdirectory, put its parent on include_path. Never point the test bootstrap to production secrets or production MariaDB.

6. Common assertions

```
$this->assertSame(42, $value);
$this->assertTrue($isActive);
$this->assertFalse($isDeleted);
$this->assertNull($missing);
$this->assertCount(2, $rows);
$this->assertArrayHasKey('id', $user);
$this->assertInstanceOf(UserService::class, $service);
$this->assertStringContainsString('invalid', $message);
```

One test method can execute multiple assertions. The assertion count in PHPUnit output is not the same as the number of test methods or source lines containing assert calls. Some compound assertions and helpers also count multiple checks.

7. Setup, teardown, and isolation

```
class CounterTest extends \PHPUnit\Framework\TestCase {
    private $counter;
    protected function setUp(): void {
        $this->counter = new Counter();
    }
    protected function tearDown(): void {
        $this->counter = null;
    }
    public function testStartsAtZero(): void {
        $this->assertSame(0, $this->counter->get());
    }
}
```

Each test should be independent and reproducible. Avoid ordering dependencies. Use fresh fixtures, a dedicated test database, and deterministic time or injected clocks for expiration behavior.

8. Testing exceptions

```
public function testRejectsZeroDivisor(): void {
    $this->expectException(InvalidArgumentException::class);
    $this->expectExceptionMessage('Divisor cannot be zero');
    $calculator = new Calculator();
    $calculator->divide(10, 0);
}
```

Set the expectation before calling the method that throws. Test both the failure and success paths. In PHP 7.4, catch Throwable at a top-level boundary only when appropriate; Exception alone does not catch every Error.

9. Data providers

```
/** @dataProvider sumCases */
public function testSums($a, $b, $expected): void {
    $this->assertSame($expected, $a + $b);
}
public function sumCases(): array {
    return [[1, 2, 3], [0, 0, 0], [-2, 1, -1]];
}
```

A data provider runs the same test with multiple input rows. It is useful for boundary values and input validation.

10. Mocking dependencies

```
interface Mailer { public function send($to, $message); }
class WelcomeService {
    private $mailer;
    public function __construct(Mailer $mailer) {
        $this->mailer = $mailer;
    }
    public function welcome($email) {
        return $this->mailer->send($email, 'Welcome');
    }
}
public function testWelcomeSendsEmail(): void {
    $mailer = $this->createMock(Mailer::class);
    $mailer->expects($this->once())
           ->method('send')
           ->with('user@example.test', 'Welcome')
           ->willReturn(true);
    $this->assertTrue((new WelcomeService($mailer))
                      ->welcome('user@example.test'));
}
```

Mocks verify interactions without sending real email or connecting to external systems. Prefer dependency injection over global singletons where possible.

11. ZF1 controller tests

ZF1 supplies Zend_Test_PHPUnit_ControllerTestCase, which can dispatch a controller request inside a PHPUnit test. Controller testing requires a valid ZF1 application bootstrap and correct routing. The example below is a starting template, not a drop-in test for every custom ZF1 project.

```
require_once 'Zend/Test/PHPUnit/ControllerTestCase.php';
class ApiControllerTest extends Zend_Test_PHPUnit_ControllerTestCase {
    public function setUp(): void {
        $this->bootstrap = array($this, 'appBootstrap');
        parent::setUp();
    }
    public function appBootstrap() {
```

```
        $app = new Zend_Application(
            APPLICATION_ENV,
            APPLICATION_PATH . '/configs/application.ini'
        );
        $app->bootstrap();
    }
    public function testMissingTokenIsRejected(): void {
        $this->dispatch('/api/email/getlist');
        $this->assertResponseCode(401);
    }
}
```

The example assumes /api/email/getlist is protected and maps to your application routing. If the route, authorization plugin, or response format differs, adapt the assertions. Verify whether your ZF1 version expects a callable bootstrap or application bootstrap class.

12. Functional API tests

For JSON APIs, assert both HTTP status and response shape. Ensure the test request is reset between dispatches, including headers and body. Use test-only users and roles; never rely on real employee records.

```
public function testInvalidTokenGets401(): void {
    $this->getRequest()->setHeader(
        'Authorization', 'Bearer invalid-token'
    );
    $this->dispatch('/api/email/getlist');
    $this->assertResponseCode(401);
    $body = json_decode($this->getResponse()->getBody(), true);
    $this->assertIsArray($body);
    $this->assertArrayHasKey('error', $body);
}
```

13. MariaDB integration tests

Use a dedicated database such as s3_test, separate database credentials, and explicit fixture cleanup. Transactions are useful for tests that share one connection, but can fail to isolate code that opens a second connection or commits internally. For those cases use per-test fixtures and reliable cleanup.

```
// Illustrative test DB configuration in application.ini
[testing : production]
resources.db.adapter = "Pdo_Mysql"
resources.db.params.host = "127.0.0.1"
resources.db.params.dbname = "s3_test"
resources.db.params.username = "s3_test_user"
resources.db.params.password = "TEST_ONLY_PASSWORD"
```

Do not inherit production database credentials into testing without overriding all connection fields. Verify the database name and host before destructive cleanup. Avoid TRUNCATE on shared databases.

```
public function testInsertAndFind(): void {
    $db = Zend_Db_Table::getDefaultAdapter();
    $id = null;
    try {
        $db->insert('test_items', ['name' => 'example']);
        $id = $db->lastInsertId();
        $row = $db->fetchRow(
            'SELECT * FROM test_items WHERE id = ?', [$id]
        );
        $this->assertSame('example', $row['name']);
    } finally {
        if ($id !== null) {
            $db->delete('test_items', ['id = ?' => $id]);
        }
    }
}
```

Create the test_items table in the test database first. This example assumes an auto-increment primary key and a name column.

14. JWT access-token tests

For your ZF1 JWT service, test issuance, verification, expiration, invalid signatures, wrong issuer, wrong audience, missing claims, and tampering. Keep JWT secrets test-only and never use production keys.

```
public function testInvalidJwtIsRejected(): void {
    $this->expectException(User_JwtException::class);
    $this->jwtService->verifyAccessToken('not.a.valid.jwt');
}

// For an expired token, prefer constructing a signed JWT
// with an exp in the past rather than sleeping in the test.
```

15. Refresh-token rotation tests

Integration tests should confirm: raw refresh token is never stored; A rotates to B; A cannot be used again; reuse detection revokes the relevant token family; logout revokes that family; a separate login family remains independent; expired tokens fail; disabled users cannot refresh. Concurrency tests need separate database connections/workers and should not be simulated with sequential calls alone.

16. HTTP status and exception mapping

Suggested boundaries: 400 for invalid request input; 401 for missing/invalid authentication; 403 for authenticated but forbidden; 404 for missing resources; 405 for wrong method; 500 for unexpected server errors. Map internal exceptions to safe API responses and log technical details server-side. Never send stack traces, SQL queries, secrets, or raw tokens to clients.

17. Code coverage

Coverage requires an enabled coverage driver such as Xdebug or PCOV. Xdebug 3 requires mode=coverage, set before the PHP process starts. Coverage percentage is not proof of correctness; focus on meaningful branch and failure-path tests.

```
# Example (shell)
XDEBUG_MODE=coverage php vendor/bin/phpunit \
  --coverage-html build/coverage

# Windows PowerShell
$env:XDEBUG_MODE = 'coverage'
php vendor/bin/phpunit --coverage-html build/coverage
```

18. Useful commands

```
php vendor/bin/phpunit
php vendor/bin/phpunit --testsuite Unit
php vendor/bin/phpunit --testsuite Integration
php vendor/bin/phpunit --testsuite Functional
php vendor/bin/phpunit --filter testInvalidTokenGets401
php vendor/bin/phpunit --testdox
php vendor/bin/phpunit --debug
```

19. Common troubleshooting

Class not found: verify include_path, Composer autoload, ZF1 prefix registration, and file name case on CentOS. No tests executed: check naming conventions and phpunit.xml suites. Database errors: verify test database exists and test credentials. Unexpected 302 redirect: check session/auth plugins and whether the route expects browser authentication. Unexpected 404: inspect module/controller/action routing. Unexpected 500: inspect server logs and

test bootstrap. Xdebug warning: confirm matching PHP/Xdebug build and active coverage mode.

20. Practical learning sequence

First run CalculatorTest. Next add validation and exception tests. Then test a service with a mocked dependency. Next add one MariaDB integration test. Then configure Zend_Test_PHPUnit_ControllerTestCase and test one protected API route. Finally test JWT and refresh-token lifecycle. Keep each step green before proceeding.

21. Final checklist

Use PHP 7.4-compatible PHPUnit 9.6; keep ZF1 paths correct; isolate the test database; do not use production secrets; assert success and failure paths; test permissions; preserve deterministic fixtures; clean up database state; do not hide unexpected exceptions; review coverage as a guide, not a target; run tests before deployment.

22. Reference documentation

Official PHPUnit 9.6 manual: https://docs.phpunit.de/en/9.6/

PHPUnit 9 getting started: https://phpunit.de/getting-started/phpunit-9.html

ZF1 controller testing (Rob Allen):
https://akrabat.com/unit-testing-controller-actions-with-zend_test_phpunit_controllertestcase/

This guide is educational and includes illustrative classes, routes, and database schema names. Adapt them to the real ZF1 project before executing tests.